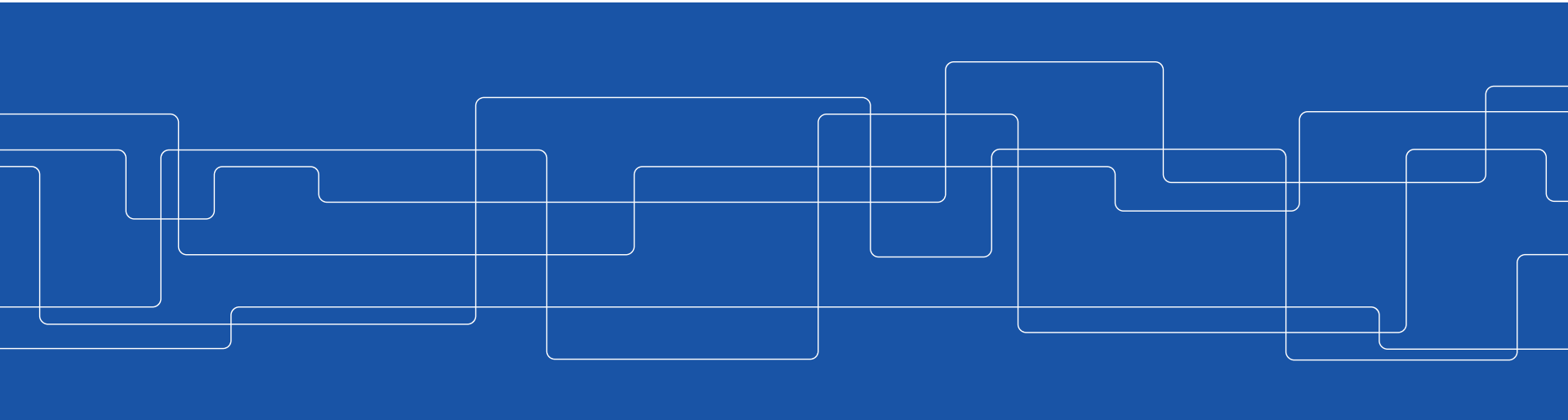




Mokav: Execution-driven Differential Testing with LLMs

Khashayar Etemadi, PhD Student at KTH

khaes@kth.se





Contents

- Introduction
- Related Work
- Mokav Approach
- Experiments
- Future Work



Introduction

The goal is to find if there is difference between the functionality of two programs P & Q

Introduction

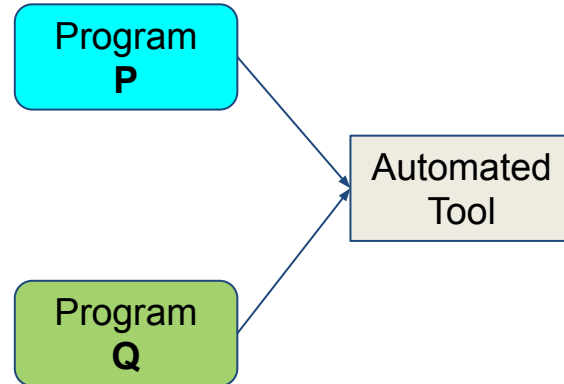
The goal is to find if there is difference between the functionality of two programs P & Q

Program
P

Program
Q

Introduction

The goal is to find if there is difference between the functionality of two programs P & Q



```
graph LR; P[Program P] --> AT[Automated Tool]; Q[Program Q] --> AT; AT --> D[Decision Equal / Different]
```

The diagram illustrates a process flow. On the left, there are two input boxes: a red box labeled "Program P" and a blue box labeled "Program Q". Arrows from both boxes point to a central yellow box labeled "Automated Tool". An arrow from the "Automated Tool" box points to a final grey box on the right labeled "Decision Equal / Different".

Introduction

The goal is to find if there is difference between the functionality of two programs P & Q

- Can prove equality with formal specification
 - A complete specification S_p can be used as follows:
 - If P & Q both verify
 - They are equal

Introduction

The goal is to find if there is difference between the functionality of two programs P & Q

- Can prove equality with formal specification

```
/*@  
  requires n > 0;  
  requires \valid_read(a + (0 .. n-1));  
  assigns \nothing;  
  ensures \forall int i; 0 <= i < n ==> a[i] <= \result;  
  ensures \exists int j; 0 <= j < n && a[j] == \result;  
*/  
int max_in_array(const int* a, int n);
```


Introduction

The goal is to find if there is difference between the functionality of two programs P & Q

- Can prove equality with formal specification
- What if we prove the difference

Introduction

The goal is to find if there is difference between the functionality of two programs P & Q

- Can prove equality with formal specification
- What if we prove the difference
 - We need to generate a difference exposing test (DET)

Introduction

The goal is to find if there is difference between the functionality of two programs P & Q

- Can prove equality with formal specification
- What if we prove the difference
 - We need to generate a difference exposing test (DET)

DET is a test input that produces different outputs on P & Q

Introduction

The goal is to find if there is difference between the functionality of two programs P & Q

- Can prove equality with formal specification
- What if we prove the difference
 - We need to generate a difference exposing test (DET)

DET is a test input that produces different outputs on P & Q

We aim at DET generation with
LLMs

Related Work

- Generating random tests
 - EvoSuite, Pynguin, Randoop, etc.

EvoSuite/**evosuite**

EvoSuite - automated generation of JUnit test suites for Java classes



randoop/**randoop**

Automatic test generation for Java



Related Work

- Generating random tests
 - EvoSuite, Pynguin, Randoop, etc.
- Fuzzing
 - AFL
 - Instruments the code
 - Computes coverage
 - Mutates test inputs to maximize coverage

EvoSuite/**evosuite**

EvoSuite - automated generation of JUnit test suites for Java classes

randoop/**randoop**

Automatic test generation for Java



Related Work

- Generating random tests
 - EvoSuite, Pynguin, Randoop, etc.
- Fuzzing
 - AFL
 - Instruments the code
 - Computes coverage
 - Mutates test inputs to maximize coverage
 - Fuzz4All, TitanFuzz
 - Use LLMs

EvoSuite/**evosuite**

EvoSuite - automated generation of JUnit test suites for Java classes



randoop/**randoop**

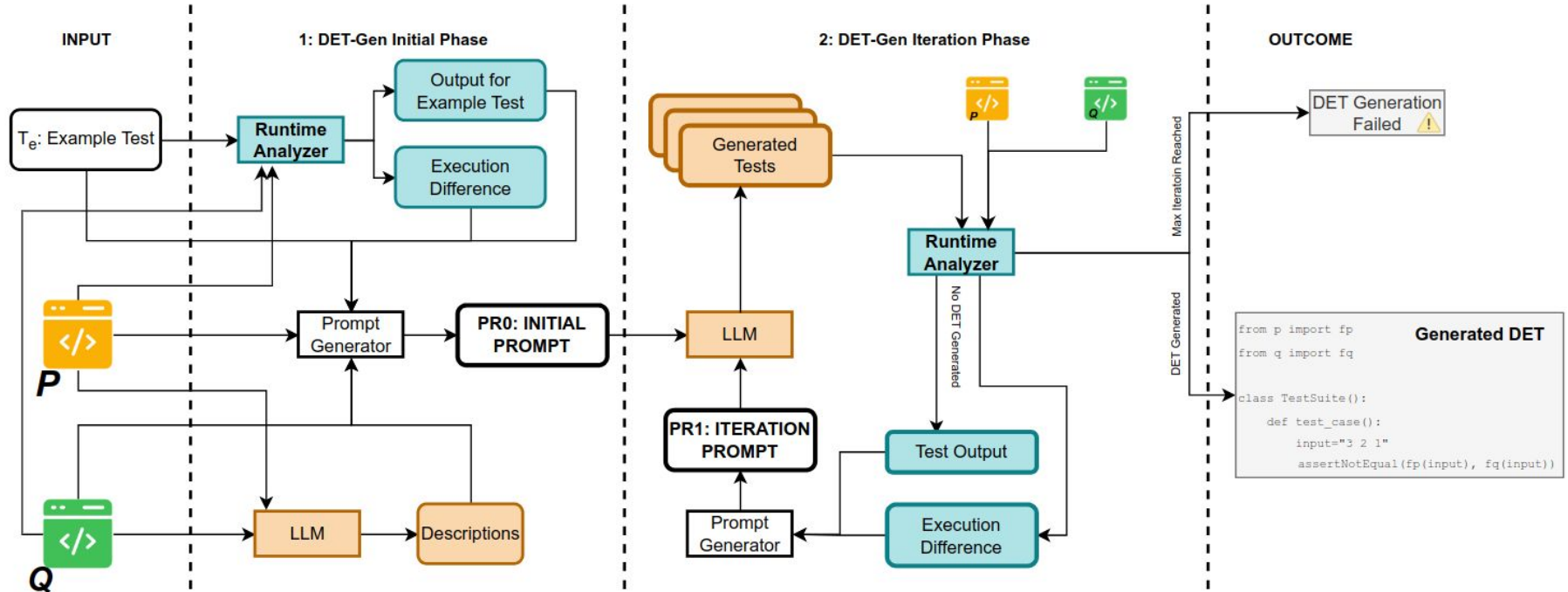
Automatic test generation for Java



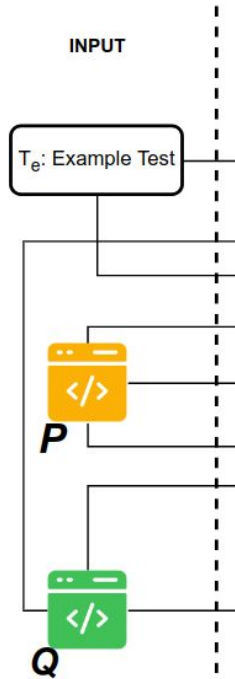
Mokav - Approach

- Mokav is an LLM-based execution-driven and iterative technique for DET generation
- The inputs are a pair of Python programs P & Q and an example test T_e
- The output is a DET

Mokav - Approach - Input



Mokav - Approach - Input





```

1 Diff between P & Q:
2 x = input().split()
3 x.sort()
4 for i in range(len(x)):
5     x[i] = int(x[i])
6 x.sort()
7 n = 100
8 for j in range(2):
9     if ((x[j] + x[(j + 1)]) > x[(j + 2)]):
10         n = 300
11     elif ((x[j] + x[(j + 1)]) == x[(j + 2)]):
12         n = max(n, 200)
13     else:
14         n = max(n, 100)
15 if (n == 300):
16     print('TRIANGLE')
17 elif (n == 200):
18     print('SIGMENT')
19     print('SEGMENT')
20 else:
21     print('IMPOSSIBLE')
22

```



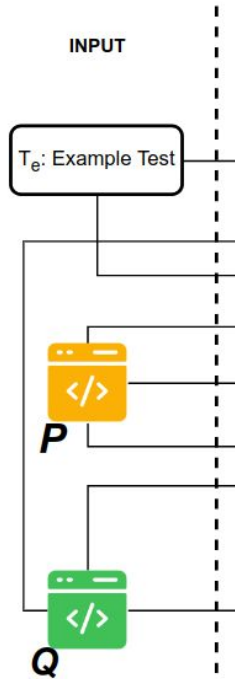
```

1 Diff between P & Q:
2 x = input().split()
3 x.sort()
4 for i in range(len(x)):
5     x[i] = int(x[i])
6 x.sort()
7 n = 100
8 for j in range(2):
9     if ((x[j] + x[(j + 1)]) > x[(j + 2)]):
10         n = 300
11     elif ((x[j] + x[(j + 1)]) == x[(j + 2)]):
12         n = max(n, 200)
13     else:
14         n = max(n, 100)
15 if (n == 300):
16     print('TRIANGLE')
17 elif (n == 200):
18     print('SEGMENT')
19     print('SEGMENT')
20 else:
21     print('IMPOSSIBLE')
22

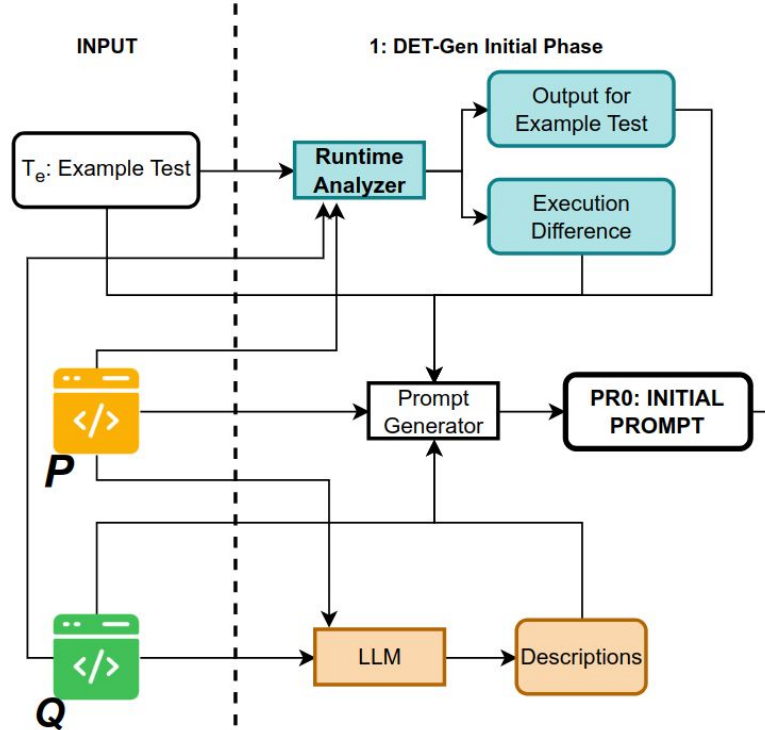
```

29

Mokav - Approach - Input



Mokav - Approach - Initial Phase





```
python
{p_version_program}
```

```
python
{q_version_program}
```

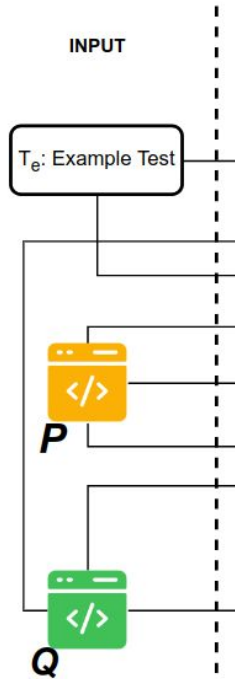
```
python
{example_test}
```

The identical output for this sample test input is `{example_test}`

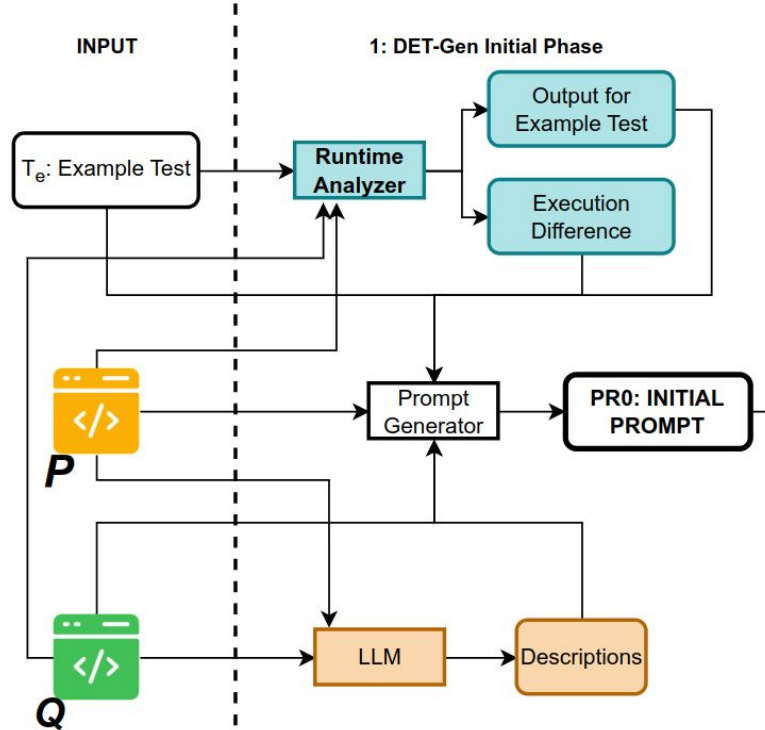


Please note that your output should not contain any explanation or newline ('\n') characters. Create a 'difference exposing test' input as per the Python dict format above.

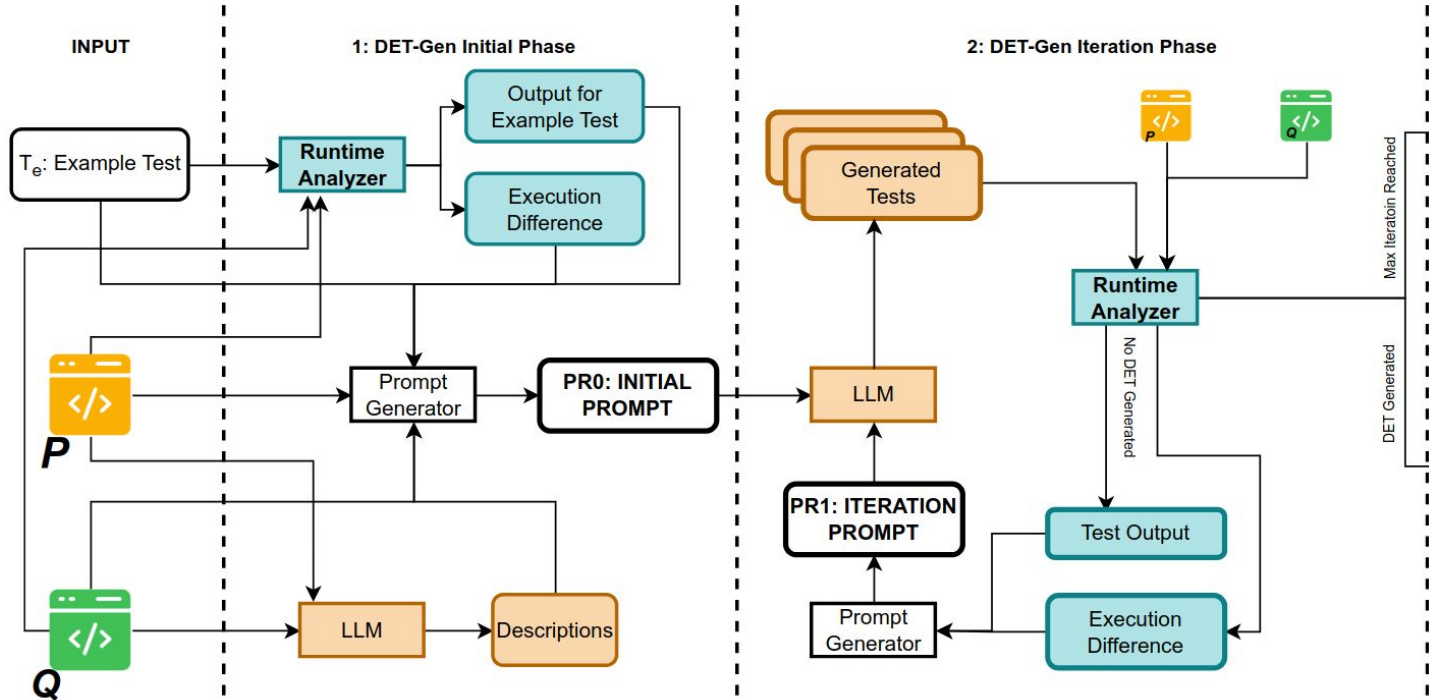
Mokav - Approach - Input



Mokav - Approach - Initial Phase



Mokav - Approach - Iteration Phase





23 **Example test:**

25 "4 2 1 3"

27 P: "TRIANGLE"

28 Q: "TRIANGLE"

29

30 **Generated Difference-exposing test (DET):**

31 Input:

32 "5 2 1 3"

33 Output:

34 P: "SIGMENT"

35 Q: "SEGMENT"



Mokav - Experiments

- Ex1 - Effectiveness
- Ex2 - Ablation study
- Ex3 - Impact of P & Q on effectiveness



Mokav - Experiments - Dataset

- Selected from Codeforces

Mokav - Experiments - Dataset

- Selected from Codeforces
- Python submissions $s_1, s_2, \dots, s_n$ by author a for problem p
 - s_i is the last rejected submission
 - s_j is the first accepted submission
 - $i < j$
 - An example test passing on both
 - Submissions and example test be small

Mokav - Experiments - Dataset

- Selected from Codeforces
- Python submissions $s_1, s_2, \dots, s_n$ by author a for problem p
 - s_i is the last rejected submission
 - s_j is the first accepted submission
 - $i < j$
 - An example test passing on both
 - Submissions and example test be small
- Result is a dataset of 1,535 program pairs + example test => C4DET

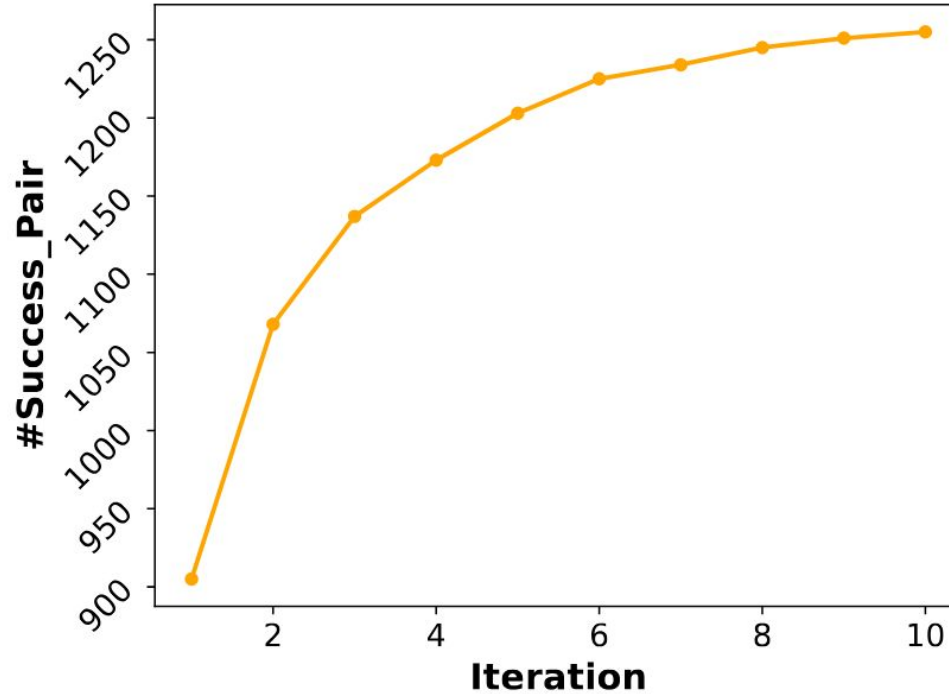
Mokav - Experiments - Dataset

- Selected from Codeforces
- Python submissions $s_1, s_2, \dots, s_n$ by author a for problem p
 - s_i is the last rejected submission
 - s_j is the first accepted submission
 - $i < j$
 - An example test passing on both
 - Submissions and example test be small
- Result is a dataset of 1,535 program pairs + example test => C4DET
- We also consider 32 pairs from QuixBugs bug fixes

Mokav - Ex1 - Effectiveness - Results

Tool	#Tests (QB)	#QuixBugs	#Tests (C4DET)	#C4DET
Pynguin	1,176	50%	27,487	18%
Mokav (it-1)	320	93%	15,350	87%
Mokav (it-4)	340	100%	30,240	90%
Mokav (it-10)	340	100%	48,930	94%

Mokav - Ex1 - Effectiveness - Results



Mokav - Ex2 - Ablation Study

- We consider various
 - Models
 - #iterations
 - #samples
 - Temperature
 - Presence of description
 - Presence of example test
 - Presence of execution difference

Mokav - Ex2 - Ablation Study

- We consider various
 - Models
 - #iterations
 - #samples
 - Temperature
 - Presence of description
 - Presence of example test
 - Presence of execution difference

**Example Test
is the most important**



```
2 # P and Q get two integers (n and k) and a string (a1) as input and return a
   list of strings as output.
```

```
5 + if ((1 <= n < 50) and (1 <= k <= 50)):
```

7

```
9 {n=50, k=1, a1='BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBGG' }
```

11 The output produced by P and Q on the generated DET:

```
13 Q output: ['BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBGB']
```

Mokav - Ex3 - Impact of P & Q on Success

No Correlation with
Example Test Size

Mokav - Ex3 - Impact of P & Q on Success

No Correlation with
Example Test Size

No Correlation with
P & Q Distance

Mokav - Ex3 - Impact of P & Q on Success

No Correlation with
Example Test Size

No Correlation with
P & Q Distance

Strongly Correlated with
P & Q Size

Mokav - Takeaway

LLMs are good at generating meaningful tests that are structurally almost correct



Mokav - Future Work

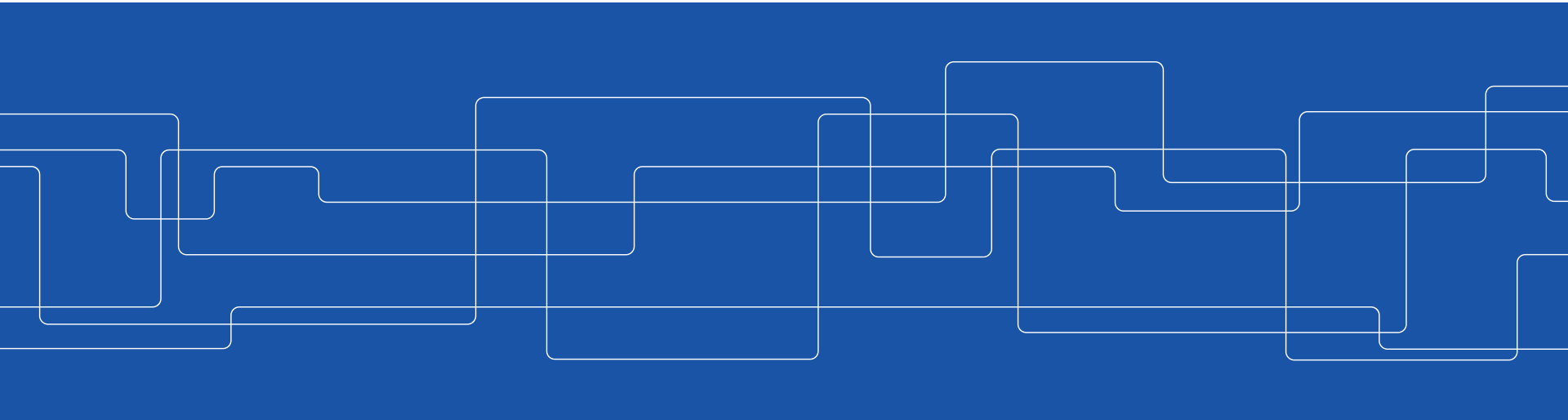
Merge LLM-based test generation with static input structure extraction



Thanks for listening!

Khashayar Etemadi

khaes@kth.se <https://github.com/ASSERT-KTH/mokav>





Q&A

